

전남대학교 인공지능혁신융합대학사업단

2026년 하반기 WE-meet 프로젝트 계획서

2026.

목차

목차	주요 내용	비고
0. 표지	프로젝트 제목	
	팀 이름	
	팀 구성원	
1. 프로젝트 개요	배경 및 목적	
	중요성 및 필요성	
2. 프로젝트 목표	구체적인 목표 설정	
	결과 및 산출물	
3. 프로젝트 범위	목표별 추진 내용	
	주요 기능 및 특징	
4. 프로젝트 방법론	연구 방법 및 절차	
	데이터 수집 및 분석	
	인공지능 모델 설계 및 개발 방법	
	주요 도구 및 기술	
	전문가 멘토링 계획	
	성과 지표 및 평가 방법	
5. 일정 및 마일스톤	프로젝트 일정	
	마일스톤 및 목표	
6. 팀 구성 및 역할	팀원의 역할	
	팀원의 주요 업무	
7. 관련 연구 및 문헌 조사	관련 문헌 및 선행연구 조사	
	기술 및 시장동향 조사	

8. 결론	프로젝트 요약 및 기대효과	
APPENDIX	참고문헌 및 정보사이트 등	

0. 표지

『전남대학교 인공지능혁신융합대학사업단』

2026 하반기 WE-Meet 프로젝트 계획서

프로젝트 제목	Prova(프로바) : 설계 문서 기반 Vision-Language 웹 GUI QA Agent 개발
팀 명	소인배

팀장	소속대학	전남대학교	성명	김동욱
	학과	인공지능학부	학년 / 학번	3 / 220057
	연락처	010-3932-5171	E-Mail	kdu5171@jnu.ac.kr

팀원	성명	학과	학년	학번	연락처	E-Mail
	김도형	인공지능학부	3	241126	010-8927-0846	rlaehgud0194@jnu.ac.kr
	이환희	인공지능학부	3	243275	010-5840-9882	leehwanhee0622@jnu.ac.kr
	윤선진	인공지능학부	1	263113	010-3762-1797	sjyundosan0223@naver.com
	조예원	인공지능학부	1	264986	010-9628-2523	dpwonnie@naver.com
	천서원	인공지능학부	1	263979	010-2506-1861	chunseowon0611@naver.com

2026년 7월 18일

대표학생 : 김동욱

김동욱

1. 프로젝트 개요

1-1. 배경 및 목적

소프트웨어 개발에서 품질보증(QA, Quality Assurance)은 서비스의 신뢰성을 좌우하는 핵심 공정이다. 특히 사용자와 직접 맞닿는 웹 GUI(Graphical User Interface)는 회원가입·로그인·검색·결제 등 핵심 기능이 정상 동작하는지를 반복적으로 검증해야 한다. 그러나 현재 대부분의 웹 QA는 여전히 사람이 직접 화면을 클릭하며 확인하는 수동 테스트에 크게 의존하고 있으며, 자동화된 경우에도 상당한 유지보수 부담을 안고 있다.

기존 GUI 테스트 자동화(Selenium, Playwright 등)는 화면 요소를 CSS Selector나 XPath와 같은 고정된 식별자(locator)로 지정한다. 이 방식은 UI 구조가 조금만 바뀌어도 — 버튼의 위치가 이동하거나, HTML 클래스명이 변경되거나, 화면 구조가 개편되면 — 테스트가 요소를 찾지 못해 대량으로 실패(broken test)한다. 실제로 QA 엔지니어들은 새로운 테스트를 작성하기보다 기존 테스트의 깨진 locator를 수정하는 유지보수에 근무 시간의 30~50%를 소모하는 것으로 보고된다. 이는 UI가 자주 변경되는 현대 애자일·데브옵스 개발 환경에서 자동화 테스트의 실효성을 떨어뜨리는 근본적 약점이다.

또한 개발 현장에서는 설계 문서(Figma 시안, 화면기획서, PDF 명세)와 실제 구현 사이의 일치 여부를 검증하는 작업 역시 수작업으로 이루어진다. 기획자가 정의한 화면명, UI 요소, 입력 조건, 에러 메시지, 성공 조건을 QA 담당자가 일일이 읽고 이에 맞는 테스트 케이스를 손으로 작성한다. 이 과정은 반복적이고 시간이 많이 들며, 기획-개발-QA 간 커뮤니케이션 비용을 키우는 원인이 된다.

본 프로젝트는 이러한 문제를 해결하기 위해 설계 문서를 입력하면 스스로 테스트 케이스를 생성하고, Vision-Language Model(VLM)로 화면 요소를 인식하여 실제 웹을 검증하며, UI가 바뀌어도 요소를 다시 찾아 테스트를 이어가는 자가치유형(Self-Healing) 웹 GUI QA Agent를 개발하는 것을 목적으로 한다. 이를 통해 반복적인 수동 QA 업무와 locator 유지보수 부담을 자동화하고, 설계 문서 기반의 신뢰성 높은 테스트 케이스를 자동 생성함으로써 개발 생산성과

소프트웨어 품질을 동시에 높이고자 한다.

1-2. 중요성 및 필요성

소프트웨어 테스트는 전 산업의 디지털 전환과 함께 지속적으로 성장하는 시장이다. 글로벌 소프트웨어 테스트 시장은 2025년 약 493.6억 달러 규모에서 2033년 931.5억 달러까지 성장(연평균 8.5%)할 것으로 전망되며, 그중에서도 AI 기반 테스트 자동화 시장은 2025년 88.1억 달러에서 2032년 359.6억 달러로 연평균 22.3%의 높은 성장세를 보일 것으로 예측된다. 특히 기업의 AI 테스트 도구 도입률은 2023년 약 15%에서 2027년 80%까지 급증할 것으로 전망되어, AI QA 자동화 수요가 폭발적으로 커지고 있음을 보여준다.

이러한 성장의 핵심 동인은 지능형 테스트 케이스 생성, 자가치유(Self-Healing) 스크립트, 예측적 결함 분석이다. 즉, 사람이 일일이 테스트를 만들고 고치던 방식에서, AI가 테스트를 생성하고 스스로 유지보수하는 방향으로 QA 패러다임이 이동하고 있다.

그러나 현재 상용 AI QA 도구들은 대부분 (1) 이미 구현된 화면을 학습·기록하는 방식에 머물러 설계 문서 단계에서 테스트를 자동 설계하지 못하거나, (2) HTML DOM 구조에 의존하여 시각적으로 렌더링된 화면 그 자체를 사람처럼 인식하지 못하는 한계가 있다. 본 프로젝트는 여기에 착안하여, 설계 문서(요구사항)와 실제 화면(Vision) 양쪽을 모두 이해하는 에이전트를 지향한다. 이는 값비싼 상용 솔루션 없이도 개발팀 내부의 QA 자동화 도구, 화면 설계 검증 도구, 테스트 케이스 자동 생성 도구로 확장 가능하며, 반복적 수동 QA 업무를 줄여 테스트 유지보수 비용을 절감하는 실질적 가치를 제공한다.

2. 프로젝트 목표

2-1. 구체적인 목표 설정

본 프로젝트의 목표는 설계 문서를 이해하고 실제 웹 화면을 사람처럼 인식하여 스스로 테스트를

수행하는 Vision-Language 기반 QA Agent를 개발하는 것이다. 목표는 난이도에 따라 1차(핵심 기능)와 2차(고도화)로 구분한다.

[1차 목표] 설계 문서 기반 QA Agent 구축

1. 설계 정보 자동 추출 : Figma 또는 PDF 화면 설계 문서에서 화면명, UI 요소, 입력 조건, 에러 메시지, 성공 조건을 구조화하여 추출한다.
2. Vision 기반 GUI 요소 탐지 : Vision-Language Model을 활용해 실제 렌더링된 화면 속 버튼·입력창 등 GUI 요소의 위치(좌표)를 탐지한다.
3. 테스트 케이스 자동 생성 : 추출된 설계 정보를 바탕으로 회원가입·로그인·검색 기능의 테스트 케이스를 자동 생성한다.
4. 자연어 → 테스트 시나리오 변환 : 사용자가 자연어로 요청하면 이를 실행 가능한 테스트 시나리오로 변환하는 QA Agent를 구현한다.

[2차 목표] Self-Healing GUI Test Agent 고도화

5. 자가치유(Self-Healing) 테스트 : UI 구조가 변경되어 요소를 찾지 못하더라도, VLM으로 요소를 다시 탐지하여 테스트를 중단 없이 이어가는 Self-Healing 기능을 구현한다.
6. 테스트 대상 확장 : 회원가입 외에 로그인, 검색, 상품 등록, 주문 조회 등 다양한 웹 기능으로 테스트 범위를 확장한다.
7. 실패 원인 자동 분류 : 테스트 실패 시 원인(요소 미탐지 / 입력값 오류 / 기대 결과 불일치 등)을 자동으로 분류하여 리포트에 제공한다.

2-2. 결과 및 산출물

본 프로젝트를 통해 다음과 같은 산출물을 도출한다.

1. 설계 문서 파서 모듈 — Figma API/MCP 및 PDF에서 화면 요소·조건을 구조화 추출하는 모듈

2. VLM 기반 GUI 요소 탐지 모듈 — 스크린샷 내 UI 요소의 클래스·좌표(bounding box)를 반환하는 탐지기
3. LLM 기반 테스트 시나리오 생성기 — 설계 정보/자연어 요청을 테스트 케이스로 변환
4. LangGraph 기반 QA Agent 워크플로우 — 탐지 → 실행 → 검증 → (실패 시)재탐색·재시도 → 리포트 생성의 상태 기반 파이프라인
5. 테스트 실행 리포트 생성기 — 실행 로그, 입력 데이터, 기대 결과, 실제 결과, 실패 원인 분류가 포함된 리포트
6. QA Agent 웹 데모/프로토타입 — 설계 문서 업로드 → 자동 테스트 → 결과 확인
7. 성능 평가 자료 — 요소 탐지 성공률, PASS/FAIL 판단 정확도 등 정량 평가 결과

3. 프로젝트 범위

3-1. 목표별 추진 내용

본 프로젝트는 QA Agent를 단계적으로 구축하기 위해 다음과 같은 추진 내용을 설정한다.

1) 설계 문서 정보 추출 파이프라인 구축

Figma REST API 또는 Figma MCP를 활용하여 프레임·레이어·텍스트·컴포넌트·좌표 정보를 추출하고, PDF 설계 문서의 경우 텍스트와 화면 구조를 파싱한다. 이를 바탕으로 각 화면의 테스트 조건(입력 조건, 성공/실패 조건, 에러 메시지)을 구조화된 형태로 정리한다.

2) Vision-Language 기반 GUI 요소 탐지 모듈 개발

실제 실행된 웹 화면의 스크린샷을 VLM으로 분석하여 버튼·입력창·링크 등 GUI 요소의 종류와 위치(좌표)를 탐지한다. 우선 가벼운 기본 selector(DOM 기반) 방식으로 요소를 탐지하고, selector로 찾지 못하는 경우 Vision Agent 방식으로 전환하는 하이브리드 구조를 설계한다.

3) LLM 기반 테스트 시나리오 생성 모듈 개발

추출된 설계 정보와 사용자의 자연어 요청을 분석하여 테스트 목적과 대상 기능을 파악하고,

이를 실행 가능한 테스트 케이스(입력값, 조작 순서, 기대 결과)로 변환한다.

4) LangGraph 기반 Agent 워크플로우 설계

실패 감지 → 재탐색 → 재시도 → 검증 → 리포트 생성으로 이어지는 과정을 상태 기반(state-based) 워크플로우로 구성하여, 요소를 찾지 못하거나 테스트가 실패했을 때 에이전트가 스스로 다음 행동을 결정하도록 한다.

5) Self-Healing 및 실패 원인 분류 기능 구현(2차)

UI 변경으로 기존 locator가 무효화되면 VLM 기반 탐지로 요소를 재확보하여 테스트를 지속한다. 또한 실패 유형을 자동 분류하여 개발자가 원인을 빠르게 파악할 수 있도록 한다.

3-2. 주요 기능 및 특징

- 1) 설계 문서 자동 해석 — Figma/PDF 설계 문서에서 화면명·UI요소·조건을 자동 추출·구조화
- 2) Vision 기반 요소 인식 — DOM에 의존하지 않고 렌더링된 화면 자체를 인식(VLM), UI 변경에 강건
- 3) 자연어 테스트 요청 — 코드 없이 자연어로 요청하면 테스트 시나리오로 자동 변환
- 4) 자가치유(Self-Healing) — UI가 바뀌어도 요소를 재탐색하여 테스트 중단 방지
- 5) 하이브리드 탐지 전략 — 빠른 selector 우선 → 실패 시 Vision Agent 전환으로 속도·강건성 균형
- 6) 자동 리포트 및 실패 원인 분류 — 실행 로그·입력·기대/실제 결과·실패 원인을 담은 리포트 자동 생성

4. 프로젝트 방법론

4-1. 연구 방법 및 절차

본 프로젝트는 설계 문서 입력부터 테스트 리포트 출력까지를 하나의 End-to-End 자동화

파이프라인으로 구현한다. 전체 절차는 다음 6단계로 구성된다.

- ① 설계 정보 추출 : Figma API/MCP·PDF 파싱으로 화면명·UI요소·입력조건·성공/실패조건 구조화
- ② 테스트 시나리오 생성 : LLM이 설계 정보/자연어 요청을 분석해 테스트 케이스(입력값·조작순서·기대결과)로 변환
- ③ 요소 탐지(Grounding) : 실행 화면 스크린샷에서 대상 UI 요소의 좌표를 확보(selector 우선 → 실패 시 VLM)
- ④ 테스트 실행 : Playwright로 클릭·입력 등 실제 브라우저 조작 수행
- ⑤ 결과 검증 : 성공 조건/에러 메시지 대조로 PASS/FAIL 판정
- ⑥ 자가치유·리포트 : 실패 시 재탐색·재시도(LangGraph 상태 전이), 최종 실행 로그·결과·실패 원인 리포트 생성

기존 LLM 단발성 스크립트 생성 방식과 달리, 본 프로젝트는 LangGraph 기반 상태 머신으로 "감지-판단-재시도-검증"의 순환 구조를 구성하여 UI 변경·일시적 실패에 능동적으로 대응하는 에이전트를 지향한다.

4-2. 데이터 수집 및 분석

1) 테스트용 웹 애플리케이션 확보

회원가입·로그인·검색·상품등록·주문조회 기능을 갖춘 테스트용 웹앱을 자체 구축하거나 오픈소스 데모 사이트를 활용한다. UI 변경에 대한 Self-Healing 성능을 검증하기 위해, 동일 기능에 대해 버튼 위치·클래스명·레이아웃을 의도적으로 변형한 여러 버전을 준비하여 변경 전후 테스트 통과율을 비교한다.

2) 설계 문서 데이터셋 구성

위 테스트 웹앱에 대응하는 Figma 시안과 PDF 화면기획서를 제작하여, 설계 문서 → 테스트

케이스 자동 생성의 추출 정확도를 평가할 수 있는 쌍(pair) 데이터를 확보한다.

3) 평가용 정답(Ground-Truth) 데이터 구축

GUI 요소 탐지 성능을 정량 평가하기 위해, 화면별 요소의 정답 좌표(bounding box)와 종류를 라벨링한 정답 데이터를 구축한다. 또한 각 테스트 케이스의 정답 PASS/FAIL 라벨을 부여하여 판단 정확도를 측정한다.

4-3. 인공지능 모델 설계 및 개발 방법

1) Vision-Language 기반 GUI Grounding

화면 스크린샷 속 UI 요소를 사람처럼 인식하기 위해 Vision-Language Grounding 모델을 활용한다. 대표적으로 NVIDIA의 LocateAnything-3B는 Parallel Box Decoding(PBD) 구조로 GUI 요소의 위치를 빠르고 정확하게 예측하며, GUI Grounding 벤치마크(ScreenSpot-Pro)에서 60.3 mean F1의 성능과 기존 텍스트 기반 VLM 대비 10배 이상 빠른 디코딩 속도를 보고하였다. 본 프로젝트는 이러한 grounding 모델을 도입하여 DOM 구조에 의존하지 않고도 시각 정보만으로 요소 좌표를 확보한다.

2) 하이브리드 탐지 전략 (Selector ↔ Vision Agent)

모든 요소를 VLM으로 탐지하면 정확하지만 느리다. 따라서 ① 빠른 selector 방식으로 우선 탐지 → ② 요소를 못 찾거나 UI가 변경된 경우 Vision Agent 방식으로 전환하는 하이브리드 전략을 설계하여, 속도와 강건성(robustness)의 균형을 맞춘다. 두 방식의 탐지 성공률·처리 속도를 비교 평가한다.

3) LLM 기반 시나리오 생성 및 실패 원인 분류

LLM이 설계 정보·자연어 요청으로부터 테스트 목적을 파악하고 테스트 케이스를 생성한다. 실패 시에는 로그·화면 상태를 근거로 실패 유형(요소 미탐지 / 입력값 오류 / 기대 결과 불일치 등)을 분류한다.

4) LangGraph 기반 Agent Workflow

실패 감지·재탐색·재시도·검증·리포트 생성을 노드(node)로 정의하고, 상태에 따라 분기(branching)·재시도(retry)·메모리(memory)를 관리하는 상태 기반 워크플로우를 구성한다. 이를 통해 단순 스크립트 실행을 넘어 스스로 판단·복구하는 에이전트 동작을 구현한다.

5) 로컬 우선(Local-first) 모델 운영 전략

학생 주도의 AI 기술 내재화라는 WE-Meet 취지와 설계 문서·화면 데이터의 프라이버시 요구에 맞춰, 로컬 소형 언어모델을 기본값으로 삼고 외부 API는 fallback으로 사용하는 온프레미스 지향 전략을 채택한다. 이미 GUI 요소 탐지(VLM)를 로컬로 수행하므로, 설계 정보 추출·시나리오 생성·실패 분류용 LLM도 로컬에 co-locate한다. 구조화된 JSON 출력은 제약 디코딩(constrained decoding)으로 강제하며, 성능이 부족한 단계는 정답 데이터셋 기반 파인튜닝(LoRA)과 디스틸레이션(외부 API로 정답 라벨을 생성해 로컬 모델을 학습)으로 끌어올린다. 정량 평가 후에도 목표 정확도에 미치지 못하는 단계(예: PDF 시각 해석, 모호한 명세의 케이스 생성)에 한해서만 외부 API 추론을 사용하며, 모든 LLM 호출은 공통 인터페이스로 추상화하여 로컬↔API 백엔드를 코드 수정 없이 교체할 수 있도록 설계한다.

4-4. 주요 도구 및 기술

[설계 문서 추출] Figma REST API / Figma MCP, PDF 파서 — 화면 요소·좌표·조건 구조화 추출

[Vision Grounding] LocateAnything-3B 등 VLM — 스크린샷 기반 GUI 요소 좌표 탐지

[에이전트 오케스트레이션] LangGraph — 상태 기반 워크플로우(감지·재시도·분기·메모리)

[LLM(로컬 우선)] 로컬 소형 LLM(예: Qwen2.5-Llama-3.2 계열) — 설계 정보/자연어 → 시나리오 생성, 실패 분류 (외부 API는 fallback)

[정형 출력] 제약 디코딩(vLLM guided_json / llama.cpp GBNF / Outlines) — 로컬 모델의 JSON 스키마 강제

[모델 개선] LoRA/QLoRA 파인튜닝 · 디스틸레이션 — 정답 데이터셋으로 로컬 성능 향상

[테스트 실행] Playwright (Python) — 실제 브라우저 조작(클릭·입력·검증) 및 스크린샷 캡처

[협업/버전관리] Git / GitHub — 파트별 병렬 개발 및 코드 관리

4-5. 전문가 멘토링 계획

본 프로젝트는 기술적 완성도를 높이는 것을 넘어, 실제 개발 현장에서 사용 가능한 QA 자동화 도구로 고도화하는 것을 목표로 한다. 멘토링 초기 단계에서는 문제 정의 및 기술 검증을 중점적으로 수행하여, 기존 QA 자동화 도구가 해결하지 못한 지점(설계 문서 미연동, DOM 의존)을 우리 솔루션이 어떻게 극복하는지 논리적으로 설계한다. 중간 단계에서는 MVP 개발과 피드백 반영에 주력하며, 모든 기능을 다루기보다 회원가입 등 핵심 시나리오에서 Self-Healing이 실제로 작동함을 입증하는 데 집중한다. 최종 단계에서는 성능 평가와 활용 시나리오 도출에 힘써, 개발팀·QA 조직 관점에서 도구의 실효성과 확장 가능성을 전달한다. 멘토링은 WE-Meet 기간 중 대면 회의를 포함한 월 2회 미팅으로 운영하며, 모든 피드백은 문서화하여 차기 개발 단계에 즉시 반영한다.

4-6. 성과 지표 및 평가 방법

본 프로젝트의 성과는 다음 정량 지표로 평가한다.

1. GUI 요소 탐지 성공률 : 정답 좌표 대비 요소를 정확히 탐지한 비율(예: IoU 기준). selector 방식과 Vision Agent 방식을 비교.
2. PASS/FAIL 판단 정확도 : 각 테스트 케이스의 성공/실패를 정답 라벨과 비교한 판정 정확도.
3. 설계 문서 기반 테스트 케이스 자동 생성 품질 : 설계 문서로부터 요구사항(입력조건·성공조건 등)을 정확히 추출·반영했는지 평가.
4. Self-Healing 복구율 : UI를 변형한 뒤에도 테스트가 요소를 재탐색하여 통과한 비율(변경

전후 통과율 비교).

5. 리포트 완결성 : 실행 로그·입력 데이터·기대 결과·실제 결과·실패 원인이 리포트에 정확히 포함되는지 확인.

6. 로컬 모델 vs 외부 API 정확도 비교 : 단계별(설계 추출·시나리오 생성·실패 분류) 정확도를 정답 데이터셋으로 비교하여, 각 단계의 로컬/API 배치를 정량적 근거로 확정한다.

5. 일정 및 마일스톤

5-1. 프로젝트 일정

7월 (착수기) : 프로젝트 킥오프(7/1), 역할 분담 확정, 개발환경 구축, 멘토 첫 미팅. 문제 정의·기술 조사 및 요구사항 정리.

8월 (기술 검증) : 테스트용 웹앱·설계 문서 데이터셋 준비, VLM/LangGraph/Playwright 기술 검증(PoC).

9월 (핵심 기능 구현 착수) : 설계 문서 추출 모듈 + LLM 테스트 시나리오 생성 모듈 개발 시작.

10월 (핵심 기능 완성) : selector 기반 실행·검증 파이프라인 완성(1차 목표). 회원가입 시나리오 End-to-End 동작 확보.

11월 (고도화) : VLM 기반 요소 탐지 결합, Self-Healing 워크플로우(LangGraph) 구현, 로그인·검색 등 대상 확장(2차 목표). 실패 원인 자동 분류 추가.

12월 (검증·마무리) : 정답 데이터 기반 정량 평가(탐지 성공률·판단 정확도·Self-Healing 복구율), 프로토타입 안정화, 멘토 자문 반영, 최종 결과보고서 작성.

5-2. 마일스톤 및 목표

M1 (8월 말) 기획·환경 구축 완료 : 테스트 웹앱·설계 문서·정답 데이터셋 확보, 기술 스택 PoC

검증.

M2 (11월 말) 핵심 시스템 통합 : 설계문서→테스트생성→실행→검증 파이프라인 + Self-Healing 통합. 회원가입·로그인·검색 시나리오 동작.

M3 (12월 말) 성능 검증·최종 보고 : 정량 성능 목표 달성 및 최종 결과보고서 제출. (정량 목표: 요소 탐지 성공률 90% 이상, PASS/FAIL 판단 정확도 90% 이상)

6. 팀 구성 및 역할

6-1. 팀원의 역할

본 프로젝트에서 팀원 6명은 전원 인공지능학부 소속으로, 설계 문서 처리 · Vision 기반 테스트 실행 · 에이전트/평가의 세 개 파트에 각 2명씩 배치되어 협업한다. 파트별 담당자는 ① 설계 문서 처리 파트에 김동욱(팀장)·윤선진, ② Vision·테스트 실행 파트에 김도형·조예원, ③ 에이전트·평가 파트에 이환희·천서원으로 구성하며, 각 파트는 3학년 1명과 1학년 1명을 짝지어 경험을 균형 있게 배치하였다. 각 파트는 독립적으로 모듈을 개발하되, LangGraph 기반 에이전트 파이프라인으로 하나의 시스템에 통합한다. 팀장(김동욱)은 자신의 파트 업무와 함께 전체 아키텍처 설계와 파트 간 인터페이스 조율을 총괄한다.

파트	담당자	인원	담당 영역
① 설계 문서 처리 파트	김동욱(팀장) 윤선진	2명	Figma API/MCP·PDF 파싱, 화면 요소·입력조건·성공/실패조건 구조화 추출, LLM 테스트 시나리오 생성
② Vision·테스트 실행	김도형	2명	VLM 기반 GUI 요소

파트	조예원		탐지, selector↔Vision 하이브리드 전략, Playwright 실행·검증
③ 에이전트·평가 파트	이환희 천서원	2명	LangGraph Self- Healing 워크플로우, 실패 원인 자동 분류·리포트, 테스트 웹앱·정답 데이터셋 구축 및 성능 평가

6-2. 팀원의 주요 업무

1. [① 김동욱·윤선진] Figma API/MCP·PDF에서 화면 요소·조건을 구조화 추출하는 설계 문서 파서 개발
2. [① 김동욱·윤선진] LLM 기반 테스트 시나리오 생성 로직 구현
3. [② 김도형·조예원] VLM(LocateAnything-3B 등)을 활용한 GUI 요소 탐지 모듈 개발 및 selector 방식과의 비교 평가
4. [② 김도형·조예원] Playwright 기반 테스트 실행·검증 및 selector↔Vision 하이브리드 탐지 전략 구현
5. [③ 이환희·천서원] LangGraph 기반 Self-Healing 에이전트 워크플로우 설계·구현 및 실패 원인 자동 분류
6. [③ 이환희·천서원] 테스트 웹앱·설계 문서·정답 데이터셋 구축, 리포트 생성 및 시스템 성능 검증

7. 관련 연구 및 문헌 조사

7-1. 관련 문헌 및 선행연구 조사

1) LLM/에이전트 기반 GUI 테스트 자동화

최근 대형언어모델(LLM)과 에이전트 오케스트레이션 기술의 발전으로, 자연어 테스트 목표를 이해하여 Playwright 스크립트를 생성하고 실패 시 로그를 분석해 스스로 수정하는 자율 QA 에이전트 연구가 활발하다. LangGraph는 메모리·재시도·분기 로직을 갖춘 상태 기반 에이전트를 구성할 수 있어 다단계 테스트 워크플로우에 적합한 프레임워크로 주목받는다. 학계에서도 LLM을 활용한 테스트 케이스 생성부터 실행·리포팅까지 자동화하는 연구(예: arXiv 2501.00217)와, 다중 에이전트(Test Engineer-HTML Interpreter-Code Executor) 구조로 테스트를 수행하는 연구(Kashef) 등이 보고되고 있다.

2) Vision-Language 기반 GUI Grounding

DOM에 의존하지 않고 화면 자체를 인식하는 GUI Grounding 연구가 빠르게 발전하고 있다. NVIDIA의 LocateAnything-3B는 Parallel Box Decoding으로 GUI 요소 위치를 고속·고정밀 예측하며 ScreenSpot-Pro 벤치마크에서 최신 성능(60.3 mean F1)을 달성하였다. 이러한 시각 기반 요소 인식은 UI 구조 변경에 강건한 테스트 자동화의 핵심 기술 기반이 된다.

3) Self-Healing 테스트 자동화

Self-Healing 테스트는 테스트가 locator 문제로 실패하려 할 때 이를 실시간으로 감지·복구하는 기법이다. 요소의 "의도(intent)"와 "구현 세부사항"을 분리하고, 구현이 바뀌면 AI가 그 간극을 메운다. 이를 통해 사소한 UI 변경으로 파이프라인이 깨지는 문제를 줄이고, 유지보수 시간을 30~40% 절감하는 것으로 보고된다. 본 프로젝트는 이 개념을 VLM 기반 재탐색으로 구현한다.

7-2. 기술 및 시장동향 조사

소프트웨어 테스트 시장은 2025년 약 493.6억 달러에서 2033년 931.5억 달러로 성장(CAGR 8.5%)할 전망이며, 그중 AI 테스트 자동화 시장은 2025년 88.1억 달러에서 2032년 359.6억 달러로 연평균 22.3% 성장할 것으로 예측된다. 기업의 AI 테스트 도구 도입률은 2023년 약

15%에서 2027년 80%로 급증할 전망으로, 지능형 테스트 생성·자가치유·예측적 결함 분석 수요가 커지고 있다.

실제 시장에서는 AI 기반 QA 도구가 이미 상용화되어 있다. Applitools는 Visual AI로 사람 눈처럼 시각적 회귀(레이아웃 이동·요소 어긋남)를 탐지하고, Mabl은 자연어 요구사항·유저스토리로부터 테스트를 자율 생성하는 에이전틱 테스트를 제공하며, Testim(Tricentis)은 애플리케이션 변경 시 테스트를 안정적으로 유지하는 self-healing에 강점이 있다. 이는 AI 기반 GUI 테스트가 이미 시장에서 구현 가능성을 보여주는 사례다.

그러나 이들 도구는 대체로 (1) 이미 구현된 화면을 기록·학습하는 방식이라 설계 문서 단계의 테스트 자동 설계를 지원하지 않고, (2) 상당수가 DOM 기반이라 렌더링된 화면 자체를 시각적으로 인식하지 못한다. 본 프로젝트는 설계 문서(요구사항)와 실제 화면(Vision)을 함께 이해하는 에이전트로 이 공백을 겨냥한다는 점에서 차별성을 가진다.

8. 결론

8-1. 프로젝트 요약 및 기대효과

[프로젝트 요약]

본 프로젝트는 웹 GUI QA의 반복적 수동 검증과 locator 유지보수 부담, 그리고 설계 문서와 구현 간 검증의 수작업 문제를 해결하기 위해 설계 문서 기반 Vision-Language 웹 GUI QA Agent 'Prova(프로바)'를 개발한다. 'Prova'는 이탈리아어로 '시험·증명'을 뜻하며, '구현이 설계대로 동작하는지 스스로 시험하여 증명하고, UI 변경에도 요소를 재탐색해 검증을 이어간다'는 프로젝트 철학을 담는다. 본 에이전트는 Figma/PDF 설계 문서에서 화면 요소·조건을 추출하고, LLM으로 테스트 시나리오를 생성하며, Vision-Language Grounding 모델로 화면 요소를 인식하여 Playwright로 실제 웹을 검증한다. 나아가 LangGraph 기반 상태 워크플로우로 UI 변경 시 요소를 재탐색하는 Self-Healing과 실패 원인 자동 분류를 수행하여, 사람의 개입

없이 테스트를 생성·실행·복구·리포팅하는 End-to-End QA 자동화를 구현한다.

[기대효과]

기술적 측면 : DOM에 의존하지 않는 Vision 기반 요소 인식과 설계 문서 연동을 결합하여, UI 변경에 강건하고 요구사항에 기반한 신뢰성 높은 테스트 자동화 파이프라인을 입증한다.

selector·Vision 하이브리드 전략으로 속도와 강건성을 동시에 확보한다.

산업적·생산성 측면 : QA 엔지니어가 근무 시간의 30~50%를 소모하던 테스트 유지보수를 자동화하여 개발 생산성을 높이고, 값비싼 상용 솔루션 없이 내부 QA 자동화·화면 설계 검증·테스트 케이스 생성 도구로 확장 가능하다. 급성장하는 AI 테스트 자동화 시장(CAGR 22%대)에서 활용도가 높은 솔루션이다.

협업적 측면 : 설계 문서를 곧바로 테스트로 연결함으로써 기획-개발-QA 간 커뮤니케이션 비용을 줄이고, 반복적 수동 QA 업무를 절감하여 팀이 핵심 개발에 집중하도록 돕는다.

APPENDIX — 참고문헌 및 정보사이트

참고 기술 문서 (주제개요서 제공)

- LocateAnything-3B : <https://huggingface.co/nvidia/LocateAnything-3B>
- LangGraph : <https://docs.langchain.com/oss/python/langgraph/overview>
- Figma REST API : <https://developers.figma.com/docs/rest-api/>
- Playwright Python : <https://playwright.dev/python/docs/intro>

선행연구 (7-1)

- LocateAnything: Fast and High-Quality Vision-Language Grounding with Parallel Box Decoding. NVIDIA Research, 2026. <https://research.nvidia.com/labs/lpr/locate-anything/>
- The Potential of LLMs in Automating Software Testing: From Generation to Reporting.

arXiv:2501.00217, 2025. <https://arxiv.org/html/2501.00217v1>

· Autonomous QA Testing With Playwright, LangGraph, and GPT-4o. DZone, 2025.

시장·기술동향 (7-2)

· Software Testing Market Size & Share, Industry Report 2033. Grand View Research.

<https://www.grandviewresearch.com/industry-analysis/software-testing-market-report>

· AI Test Automation Market worth \$35.96 billion by 2032. MarketsandMarkets.

<https://www.marketsandmarkets.com/PressReleases/ai-test-automation.asp>

· Self-Healing Test Automation: A Practical Guide. Katalon. <https://katalon.com/resources-center/blog/self-healing-test-automation>

· Best AI Testing Tools (Applitools, Mabl, Testim 등) 비교, 2026.